

HAS-A can be better than IS-A

The HAS-A relationship is an interesting one: each duck has a FlyBehavior and a QuackBehavior to which it delegates flying and quacking.

When you put two classes together like this you're using **composition**. Instead of *inheriting* their behavior, the ducks get their behavior by being *composed* with the right behavior object.

This is an important technique; in fact, it is the basis of our third design principle:



As you've seen, creating systems using composition gives you a lot more flexibility. Not only does it let you encapsulate a family of algorithms into their own set of classes, but it also lets you **change behavior at runtime** as long as the object you're composing with implements the correct behavior interface.

Composition is used in *many* design patterns and you'll see a lot more about its advantages and disadvantages throughout the book.



A duck call is a device that hunters use to mimic the calls (quacks) of ducks. How would you implement your own duck call that does *not* inherit from the Duck class?



Guru and Student...

Guru: Tell me what you have learned of the Object-Oriented ways.

Student: Guru, I have learned that the promise of the object-oriented way is reuse.

Guru: Continue...

Student: Guru, through inheritance all good things may be reused and so we come to drastically cut development time like we swiftly cut bamboo in the woods.

Guru: Is more time spent on code **before** or **after** development is complete?

Student: The answer is **after**, Guru. We always spend more time maintaining and changing software than on initial development.

Guru: So, should effort go into reuse **above** maintainability and extensibility?

Student: Guru, I believe that there is truth in this.

Guru: I can see that you still have much to learn. I would like for you to go and meditate on inheritance further. As you've seen, inheritance has its problems, and there are other ways of achieving reuse.

Speaking of Design Patterns...



Congratulations on
your first pattern!

You just applied your first design pattern—the **STRATEGY** Pattern. That's right, you used the Strategy Pattern to rework the SimUDuck app.

Thanks to this pattern, the simulator is ready for any changes those execs might cook up on their next business trip to Maui.

Now that we've made you take the long road to learn it, here's the formal definition of this pattern:

The Strategy Pattern defines a family of algorithms, encapsulates each one, and makes them interchangeable. Strategy lets the algorithm vary independently from clients that use it.

Use THIS definition when you need to impress friends and influence key executives.



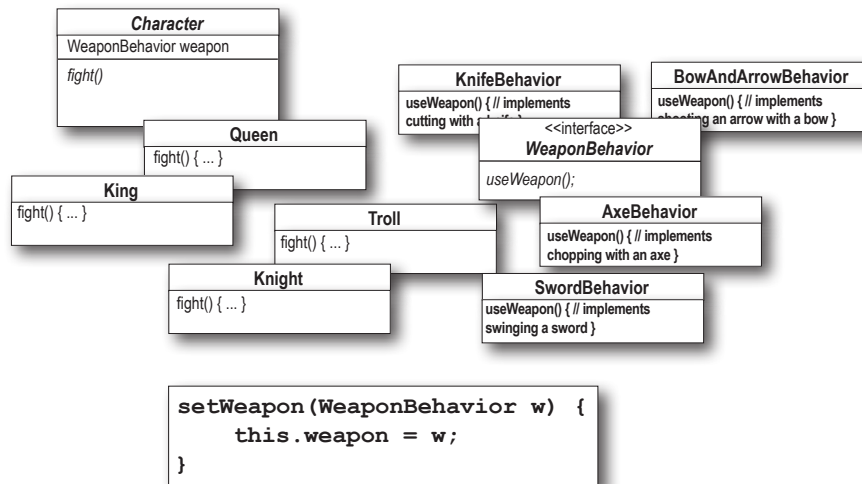
Design Puzzle

Below you'll find a mess of classes and interfaces for an action adventure game. You'll find classes for game characters along with classes for weapon behaviors the characters can use in the game. Each character can make use of one weapon at a time, but can change weapons at any time during the game. Your job is to sort it all out...

(Answers are at the end of the chapter.)

Your task:

- 1 Arrange the classes.
- 2 Identify one abstract class, one interface, and eight classes.
- 3 Draw arrows between classes.
 - a. Draw this kind of arrow for inheritance ("extends"). 
 - b. Draw this kind of arrow for interface ("implements"). 
 - c. Draw this kind of arrow for HAS-A. 
- 4 Put the method `setWeapon()` into the right class.



Overheard at the local diner...

Alice

I need a cream cheese with jelly on white bread, a chocolate soda with vanilla ice cream, a grilled cheese sandwich with bacon, a tuna fish salad on toast, a banana split with ice cream & sliced bananas, and a coffee with a cream and two sugars ... oh, and put a hamburger on the grill!

Flo

Give me a C.J. White, a black & white, a Jack Benny, a radio, a house boat, a coffee regular, and burn one!



What's the difference between these two orders? Not a thing! They're both the same order, except Alice is using twice the number of words and trying the patience of a grumpy short-order cook.

What's Flo got that Alice doesn't? **A shared vocabulary** with the short-order cook. Not only does that make it easier to communicate with the cook, but it gives the cook less to remember because he's got all the diner patterns in his head.

Design Patterns give you a shared vocabulary with other developers. Once you've got the vocabulary, you can more easily communicate with other developers and inspire those who don't know patterns to start learning them. It also elevates your thinking about architectures by letting you **think at the pattern level**, not the nitty-gritty *object* level.

Overheard in the next cubicle...

So I created this broadcast class. It keeps track of all the objects listening to it, and anytime a new piece of data comes along it sends a message to each listener. What's cool is that the listeners can join the broadcast at any time or they can even remove themselves. It is really dynamic and loosely coupled!

Rick



Can you think of other shared vocabularies that are used beyond OO design and diner talk? (Hint: how about auto mechanics, carpenters, gourmet chefs, and air traffic controllers?) What qualities are communicated along with the lingo?

Can you think of aspects of OO design that get communicated along with pattern names? What qualities get communicated along with the name "Strategy Pattern"?

Rick, why didn't you just say you are using the **Observer Pattern**?



Exactly. If you communicate in patterns, then other developers know immediately and *precisely* the design you're describing. Just don't get Pattern Fever...you'll know you have it when you start using patterns for Hello World...

The power of a shared pattern vocabulary

When you communicate using patterns, you are doing more than just sharing LINGO.

Shared pattern vocabularies are POWERFUL.

When you communicate with another developer or your team using patterns, you are communicating not just a pattern name but a whole set of qualities, characteristics, and constraints that the pattern represents.

"We're using the Strategy Pattern to implement the various behaviors of our ducks." This tells you the duck behavior has been encapsulated into its own set of classes that can be easily expanded and changed, even at runtime if needed.

Patterns allow you to say more with less. When you use a pattern in a description, other developers quickly know precisely the design you have in mind.

Talking at the pattern level allows you to stay "in the design" longer. Talking about software systems using patterns allows you to keep the discussion at the design level, without having to dive down to the nitty-gritty details of implementing objects and classes.

How many design meetings have you been in that quickly degrade into implementation details?

Shared vocabularies can turbo-charge your development team. A team well versed in design patterns can move more quickly with less room for misunderstanding.

As your team begins to share design ideas and experience in terms of patterns, you will build a community of pattern users.

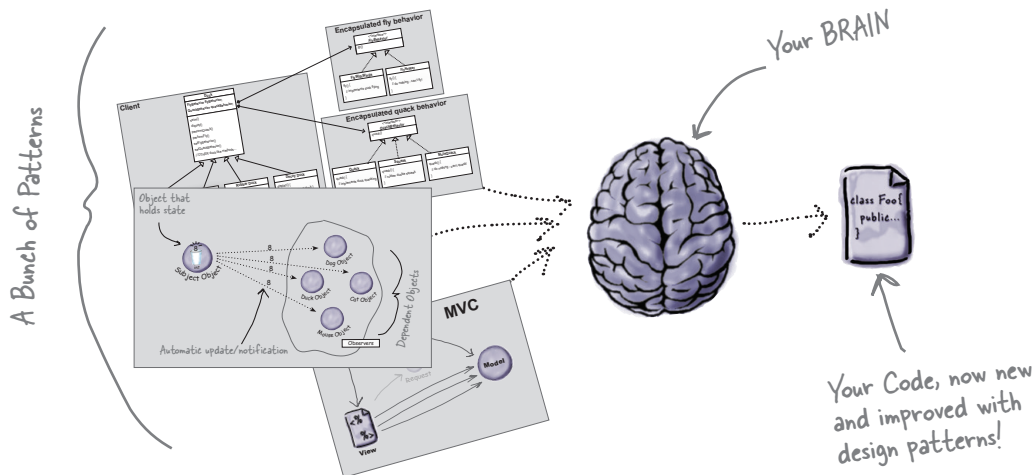
Shared vocabularies encourage more junior developers to get up to speed. Junior developers look up to experienced developers. When senior developers make use of design patterns, junior developers also become motivated to learn them. Build a community of pattern users at your organization.

Think about starting a patterns study group at your organization. Maybe you can even get paid while you're learning...

How do I use Design Patterns?

We've all used off-the-shelf libraries and frameworks. We take them, write some code against their APIs, compile them into our programs, and benefit from a lot of code someone else has written. Think about the Java APIs and all the functionality they give you: network, GUI, IO, etc. Libraries and frameworks go a long way toward a development model where we can just pick and choose components and plug them right in. But...they don't help us structure our own applications in ways that are easier to understand, more maintainable, and more flexible. That's where design patterns come in.

Design patterns don't go directly into your code, they first go into your **BRAIN**. Once you've loaded your brain with a good working knowledge of patterns, you can then start to apply them to your new designs, and rework your old code when you find it's degrading into an inflexible mess.



there are no Dumb Questions

Q: If design patterns are so great, why can't someone build a library of them so I don't have to?

A: Design patterns are higher level than libraries. Design patterns tell us how to structure classes and objects to solve certain problems, and it is our job to adapt those designs to fit our particular application.

Q: Aren't libraries and frameworks also design patterns?

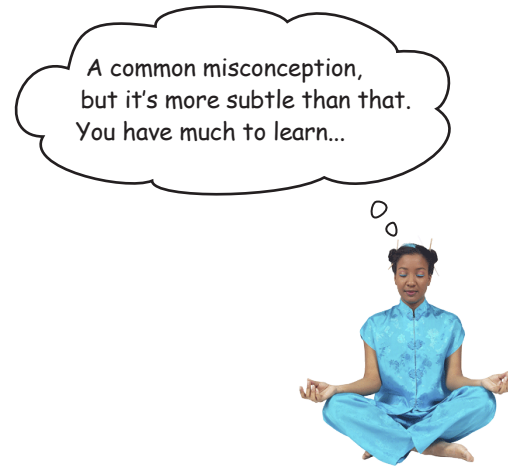
A: Frameworks and libraries are not design patterns; they provide specific implementations that we link into our code. Sometimes, however, libraries and frameworks make use of design patterns in their implementations. That's great, because once you understand design patterns, you'll more quickly understand APIs that are structured around design patterns.

Q: So, there are no libraries of design patterns?

A: No, but you will learn later about patterns catalogs with lists of patterns that you can apply to your applications.



Skeptical Developer



Friendly Patterns Guru

Developer: Okay, hmm, but isn't this all just good object-oriented design; I mean as long as I follow encapsulation and I know about abstraction, inheritance, and polymorphism, do I really need to think about Design Patterns? Isn't it pretty straightforward? Isn't this why I took all those OO courses? I think Design Patterns are useful for people who don't know good OO design.

Guru: Ah, this is one of the true misunderstandings of object-oriented development: that by knowing the OO basics we are automatically going to be good at building flexible, reusable, and maintainable systems.

Developer: No?

Guru: No. As it turns out, constructing OO systems that have these properties is not always obvious and has been discovered only through hard work.

Developer: I think I'm starting to get it. These, sometimes non-obvious, ways of constructing object-oriented systems have been collected...

Guru: ...yes, into a set of patterns called Design Patterns.

Developer: So, by knowing patterns, I can skip the hard work and jump straight to designs that always work?

Guru: Yes, to an extent, but remember, design is an art. There will always be tradeoffs. But, if you follow well-thought-out and time-tested design patterns, you'll be way ahead.

Developer: What do I do if I can't find a pattern?

Remember, knowing concepts like abstraction, inheritance, and polymorphism does not make you a good object-oriented designer. A design guru thinks about how to create flexible designs that are maintainable and can cope with change.



Guru: There are some object-oriented principles that underlie the patterns, and knowing these will help you to cope when you can't find a pattern that matches your problem.

Developer: Principles? You mean beyond abstraction, encapsulation, and...

Guru: Yes, one of the secrets to creating maintainable OO systems is thinking about how they might change in the future, and these principles address those issues.



Tools for your Design Toolbox

You've nearly made it through the first chapter! You've already put a few tools in your OO toolbox; let's make a list of them before we move on to Chapter 2.

OO Basics

Abstraction
Encapsulation
Polymorphism
Inheritance

We assume you know the OO basics like abstraction, encapsulation, polymorphism, and inheritance. If you are a little rusty on these, pull out your favorite object-oriented book and review, then skim this chapter again.

OO Principles

Encapsulate what varies.
Favor composition over inheritance.
Program to interfaces, not implementations.

We'll be taking a closer look at these down the road and also adding a few more to the list.

OO Patterns

Strategy – defines a family of algorithms, encapsulates each one, and makes them interchangeable. Strategy lets the algorithm vary independently from clients that use it.

One down, many to go!

Throughout the book, think about how patterns rely on OO basics and principles.



BULLET POINTS

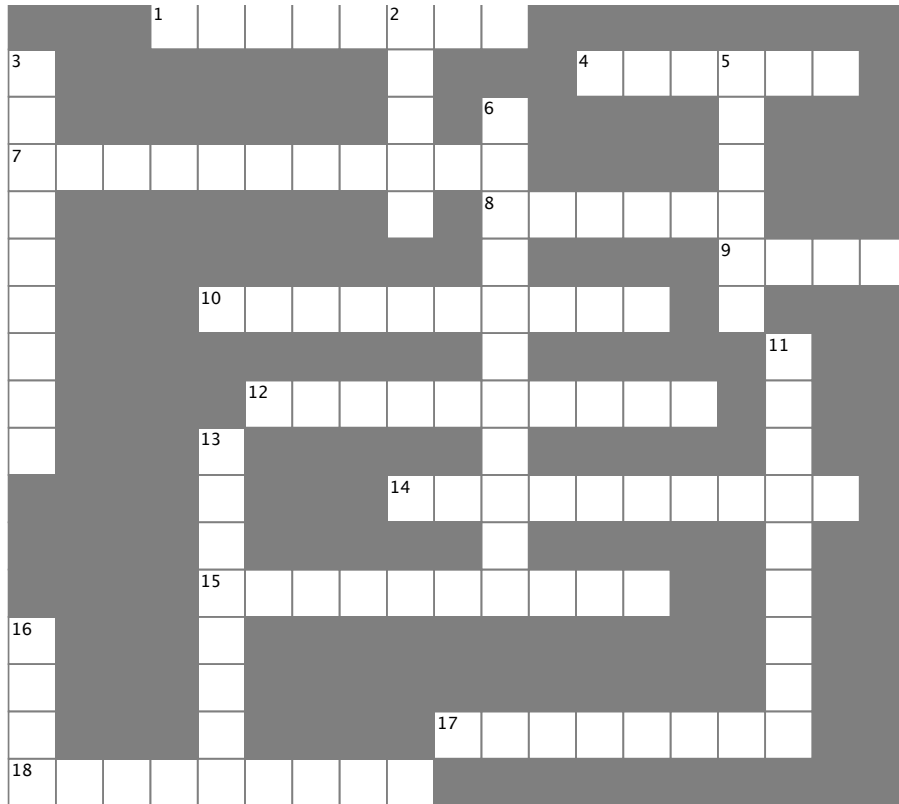
- Knowing the OO basics does not make you a good OO designer.
- Good OO designs are reusable, extensible, and maintainable.
- Patterns show you how to build systems with good OO design qualities.
- Patterns are proven object-oriented experience.
- Patterns don't give you code, they give you general solutions to design problems. You apply them to your specific application.
- Patterns aren't *invented*, they are *discovered*.
- Most patterns and principles address issues of *change* in software.
- Most patterns allow some part of a system to vary independently of all other parts.
- We often try to take what varies in a system and encapsulate it.
- Patterns provide a shared language that can maximize the value of your communication with other developers.



Design Patterns Crossword

Let's give your right brain something to do.

It's your standard crossword; all of the solution words are from this chapter.



ACROSS

1. Patterns can help us build _____ applications.
4. Strategies can be _____.
7. Favor this over inheritance.
8. Development constant.
9. Java IO, Networking, Sound.
10. Most patterns follow from OO _____.
12. Design patterns are a shared _____.
14. High-level libraries.
15. Learn from the other guy's _____.
17. Pattern that fixed the simulator.
18. Program to this, not an implementation.

DOWN

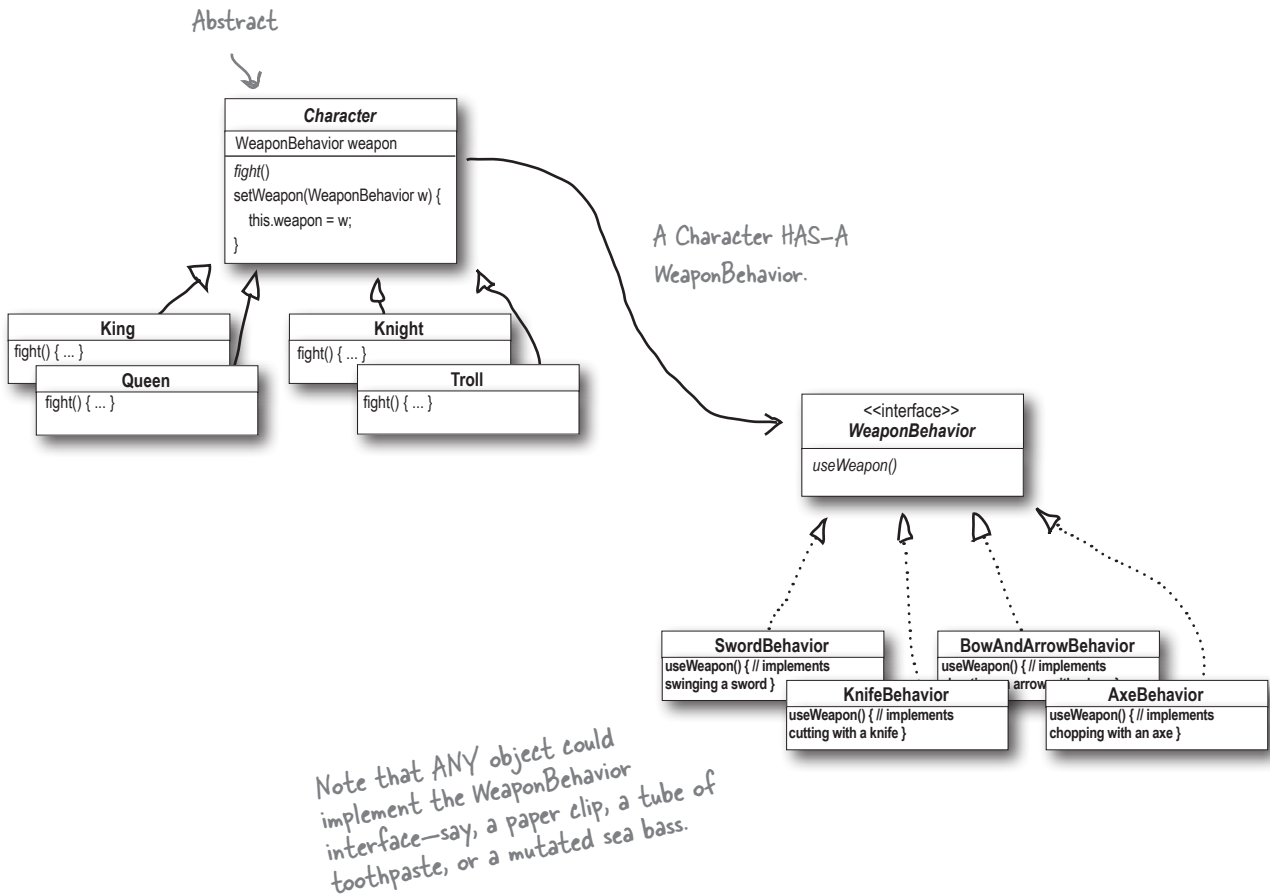
2. Patterns go into your _____.
3. Duck that can't quack.
5. Rubber ducks make a _____.
6. _____ what varies.
11. Grilled cheese with bacon.
13. Rick was thrilled with this pattern.
16. Duck demo was located here.



Design Puzzle Solution

Character is the abstract class for all the other characters (King, Queen, Knight, and Troll), while WeaponBehavior is an interface that all weapon behaviors implement. So all actual characters and weapons are concrete classes.

To switch weapons, each character calls the `setWeapon()` method, which is defined in the Character superclass. During a fight the `useWeapon()` method is called on the current weapon set for a given character to inflict great bodily damage on another character.





Sharpen your pencil Solution

Which of the following are disadvantages of using subclassing to provide specific Duck behavior? (Choose all that apply.) Here's our solution.

- | | |
|--|---|
| ☒ A. Code is duplicated across subclasses. | ☒ D. It's hard to gain knowledge of all duck behaviors. |
| ☒ B. Runtime behavior changes are difficult. | ☐ E. Ducks can't fly and quack at the same time. |
| ☐ C. We can't make ducks dance. | ☒ F. Changes can unintentionally affect other ducks. |



Sharpen your pencil Solution

What are some factors that drive change in your applications? You might have a very different list, but here's a few of ours. Look familiar? Here's our solution.

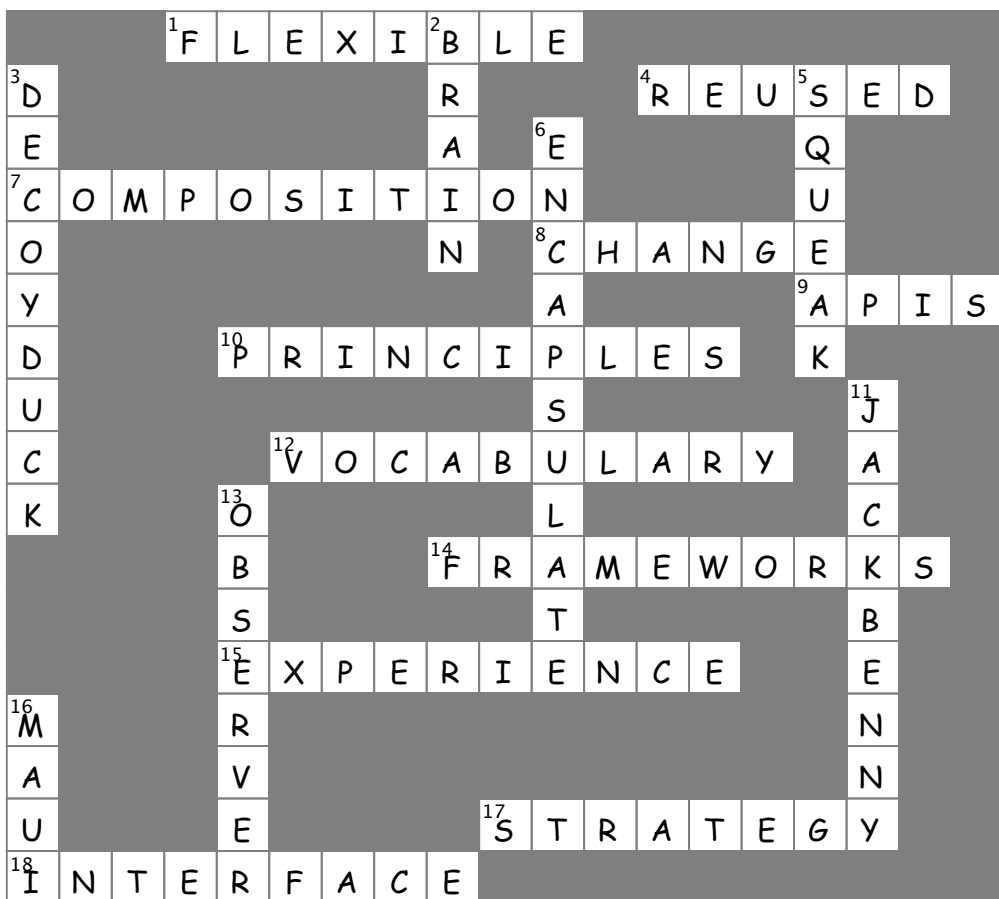
My customers or users decide they want something else, or they want new functionality.

My company decided it is going with another database vendor and it is also purchasing its data from another supplier that uses a different data format. Argh!

Well, technology changes and we've got to update our code to make use of protocols.

We've learned enough building our system that we'd like to go back and do things a little better.

Design Patterns Crossword Solution



2 the Observer Pattern

Keeping your Objects in the Know



Hey Jerry, I'm notifying everyone that the Patterns Group meeting moved to Saturday night. We're going to be talking about the Observer Pattern. That pattern is the best! It's the BEST, Jerry!

You don't want to miss out when something interesting happens, do you? We've got a pattern that keeps your objects *in the know* when something they *care about* happens. It's the Observer Pattern. It is one of the most commonly used design patterns, and it's incredibly useful. We're going to look at all kinds of interesting aspects of Observer, like its *one-to-many relationships* and *loose coupling*. And, with those concepts in mind, how can you help but be the life of the Patterns Party?

Congratulations!

Your team has just won the contract to build Weather-O-Rama, Inc.'s next-generation, internet-based Weather Monitoring Station.



Weather-O-Rama, Inc.
100 Main Street
Tornado Alley, OK 45021

Statement of Work

Congratulations on being selected to build our next-generation, internet-based Weather Monitoring Station!

The weather station will be based on our patent pending WeatherData object, which tracks current weather conditions (temperature, humidity, and barometric pressure). We'd like you to create an application that initially provides three display elements: current conditions, weather statistics, and a simple forecast, all updated in real time as the WeatherData object acquires the most recent measurements.

Further, this is an expandable weather station. Weather-O-Rama wants to allow other developers to write their own weather displays and plug them right in. So it's important that new displays will be easy to add in the future.

Weather-O-Rama thinks we have a great business model: once the customers are hooked, we intend to charge them for each display they use. Now for the best part: we are going to pay you in stock options.

We look forward to seeing your design and alpha application.

Sincerely,

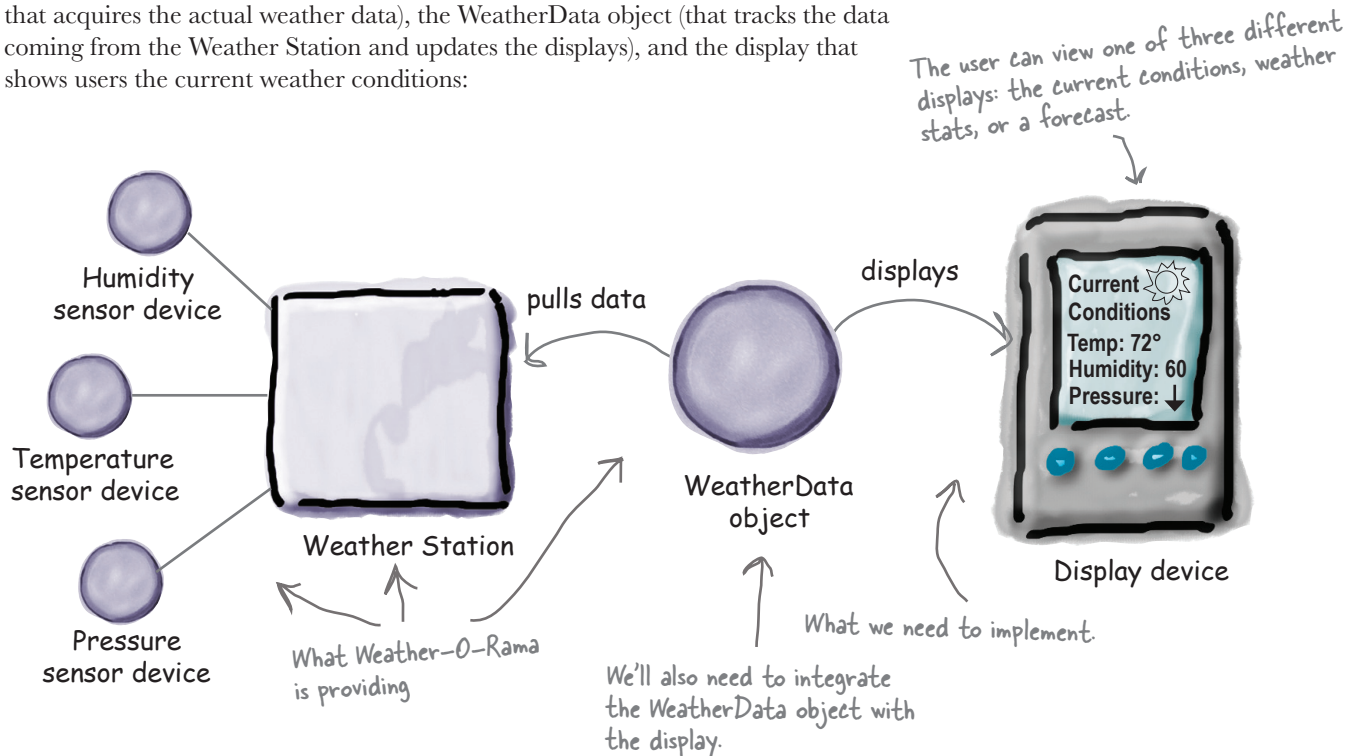
Johnny Hurricane

Johnny Hurricane, CEO

P.S. See the attached WeatherData source files!

The Weather Monitoring application overview

Let's take a look at the Weather Monitoring application we need to deliver—both what Weather-O-Rama is giving us, and what we're going to need to build or extend. The system has three components: the weather station (the physical device that acquires the actual weather data), the WeatherData object (that tracks the data coming from the Weather Station and updates the displays), and the display that shows users the current weather conditions:



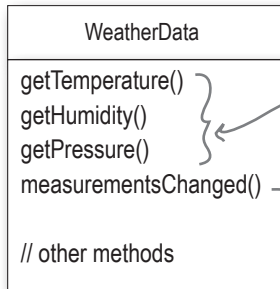
The WeatherData object was written by Weather-O-Rama and knows how to talk to the physical Weather Station to get updated weather data. We'll need to adapt the WeatherData object so that it knows how to update the display. Hopefully Weather-O-Rama has given us hints for how to do this in the source code. Remember, we're responsible for implementing three different display elements: Current Conditions (shows temperature, humidity, and pressure), Weather Statistics, and a simple Forecast.

So, our job, if we choose to accept it, is to create an app that uses the WeatherData object to update three displays for current conditions, weather stats, and a forecast.

Unpacking the WeatherData class

Let's check out the source code attachments that Johnny Hurricane, the CEO, sent over. We'll start with the WeatherData class:

Here is our WeatherData class.



These three methods return the most recent weather measurements for temperature, humidity, and barometric pressure, respectively. We don't care right now HOW it gets this data, we just know that the WeatherData object gets updated info from the Weather Station. Note that whenever WeatherData has updated values, the measurementsChanged() method is called.

Let's look at the measurementsChanged() method, which, again, gets called anytime the WeatherData obtains new values for temp, humidity, and pressure.

```
/*
 * This method gets called
 * whenever the weather measurements
 * have been updated
 *
 */
public void measurementsChanged() {
    // Your code goes here
}
```

WeatherData.java

Our soon-to-be-implemented display.



Display device

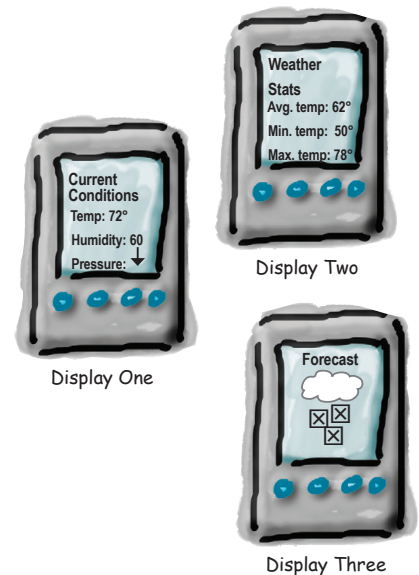
It looks like Weather-O-Rama left a note in the comments to add our code here. So perhaps this is where we need to update the display (once we've implemented it)

So, our job is to alter the measurementsChanged() method so that it updates the three displays for current conditions, weather stats, and forecast.

Our Goal

We know we need to implement a display and then have the WeatherData update that display each time it has new values, or, in other words, each time the `measurementsChanged()` method is called. But how? Let's think through what we're trying to achieve:

- We know the WeatherData class has getter methods for three measurement values: temperature, humidity, and barometric pressure.
- We know the `measurementsChanged()` method is called anytime new weather measurement data is available. (Again, we don't know or care how this method is called; we just know that it *is called*.)
- We'll need to implement three display elements that use the weather data: a *current conditions* display, a *statistics display*, and a *forecast* display. These displays must be updated as often as the WeatherData has new measurements.
- To update the displays, we'll add code to the `measurementsChanged()` method.



Stretch Goal

But let's also think about the future—remember the constant in software development? Change. We expect, if the Weather Station is successful, there will be more than three displays in the future, so why not create a marketplace for additional displays? So, how about we build in:

- **Expandability**—other developers may want to create new custom displays. Why not allow users to add (or remove) as many display elements as they want to the application? Currently, we know about the initial *three* display types (current conditions, statistics, and forecast), but we expect a vibrant marketplace for new displays in the future.



Taking a first, misguided implementation of the Weather Station

Here's a first implementation possibility—as we've discussed, we're going to add our code to the `measurementsChanged()` method in the `WeatherData` class:

```
public class WeatherData {
```

```
    // instance variable declarations
```

```
    public void measurementsChanged() {
```

```
        float temp = getTemperature();
        float humidity = getHumidity();
        float pressure = getPressure();
```

```
        currentConditionsDisplay.update(temp, humidity, pressure);
        statisticsDisplay.update(temp, humidity, pressure);
        forecastDisplay.update(temp, humidity, pressure);
```

```
    }
```

```
    // other WeatherData methods here
```

```
}
```

Here's the `measurementsChanged()` method.

And here are our code additions...

First, we grab the most recent measurements by calling the `WeatherData`'s getter methods. We assign each value to an appropriately named variable.

Next we're going to update each display...

...by calling its update method and passing it the most recent measurements.



Sharpen your pencil

Based on our first implementation, which of the following apply?
(Choose all that apply.)

- | | |
|--|---|
| ☐ A. We are coding to concrete implementations, not interfaces. | ☐ D. The display elements don't implement a common interface. |
| ☐ B. For every new display we'll need to alter this code. | ☐ E. We haven't encapsulated the part that changes. |
| ☐ C. We have no way to add (or remove) display elements at runtime. | ☐ F. We are violating encapsulation of the <code>WeatherData</code> class. |

What's wrong with our implementation anyway?

Think back to all those Chapter 1 concepts and principles—which are we violating, and which are we not? Think in particular about the effects of change on this code. Let's work through our thinking as we look at the code:

```
public void measurementsChanged() {
```

Let's take another look...

```
    float temp = getTemperature();
    float humidity = getHumidity();
    float pressure = getPressure();
```

Looks like an area of change. We need to encapsulate this.

```
    currentConditionsDisplay.update(temp, humidity, pressure);
    statisticsDisplay.update(temp, humidity, pressure);
    forecastDisplay.update(temp, humidity, pressure);
}
```

By coding to concrete implementations, we have no way to add or remove other display elements without making changes to the code.

At least we seem to be using a common interface to talk to the display elements...they all have an `update()` method that takes the temp, humidity, and pressure values.

What if we want to add or remove displays at runtime? This looks hardcoded.

Umm, I know I'm new here, but given that we are in the Observer Pattern chapter, maybe we should start using it?



Good idea. Let's take a look at Observer, then come back and figure out how to apply it to the Weather Monitoring app.

Meet the Observer Pattern

You know how newspaper or magazine subscriptions work:

- ❶ A newspaper publisher goes into business and begins publishing newspapers.
- ❷ You subscribe to a particular publisher, and every time there's a new edition it gets delivered to you. As long as you remain a subscriber, you get new newspapers.
- ❸ You unsubscribe when you don't want papers anymore, and they stop being delivered.
- ❹ While the publisher remains in business, people, hotels, airlines, and other businesses constantly subscribe and unsubscribe to the newspaper.

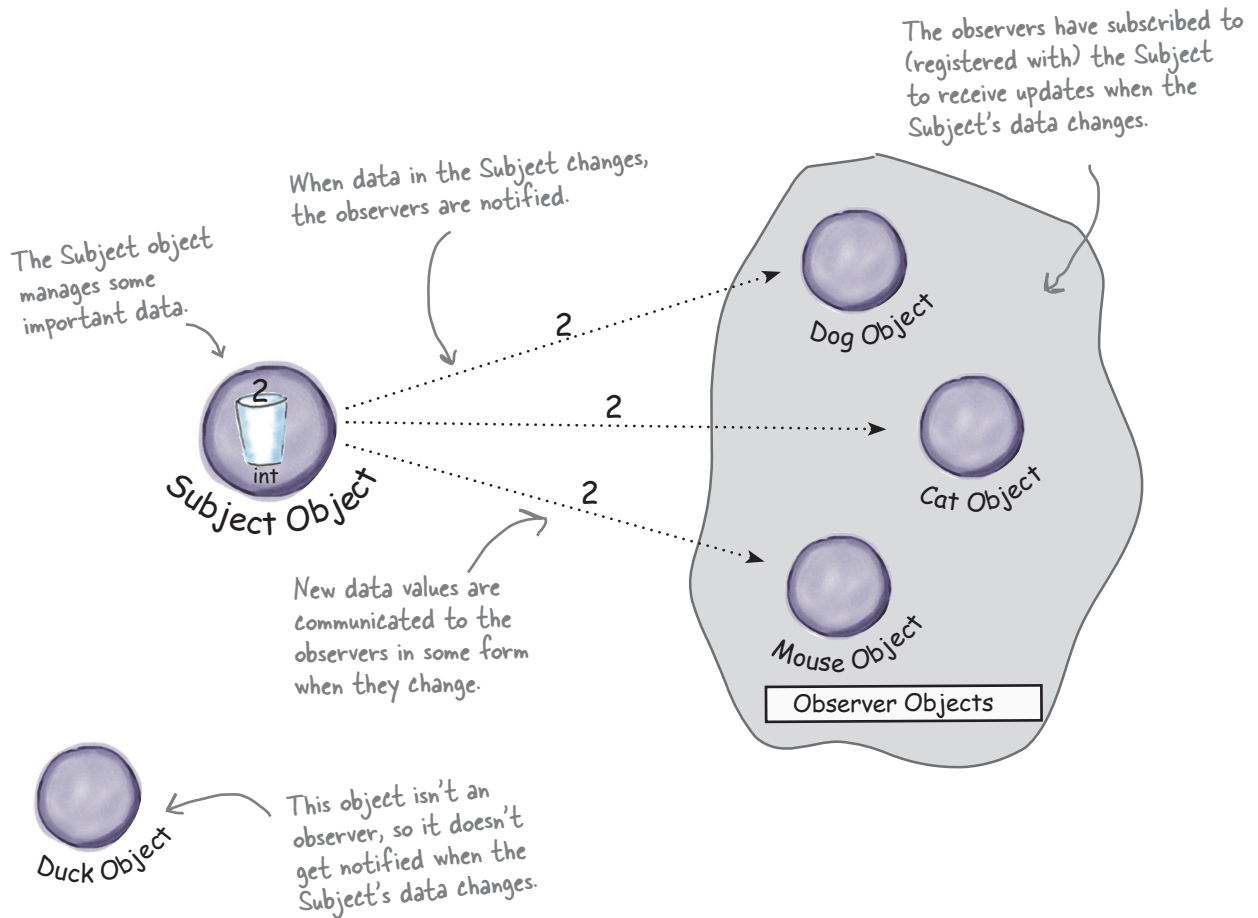
No way we want to miss what's going on in Objectville. Of course we subscribe.



Publishers + Subscribers = Observer Pattern

If you understand newspaper subscriptions, you pretty much understand the Observer Pattern, only we call the publisher the **SUBJECT** and the subscribers the **OBSERVERS**.

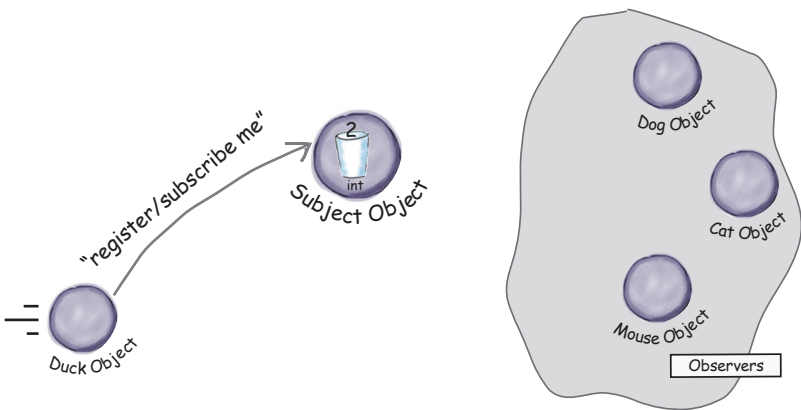
Let's take a closer look:



A day in the life of the Observer Pattern

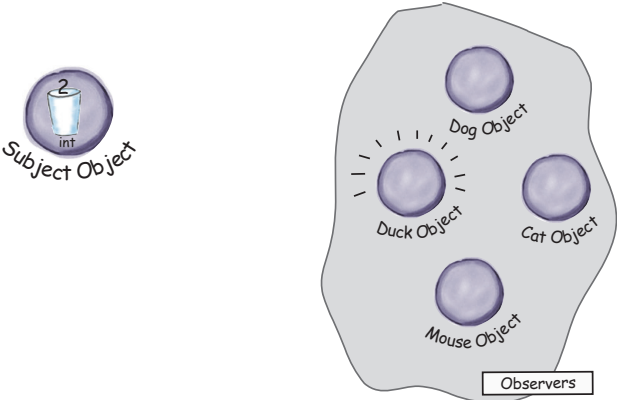
A Duck object comes along and tells the Subject that he wants to become an observer.

Duck really wants in on the action; those ints Subject is sending out whenever its state changes look pretty interesting...



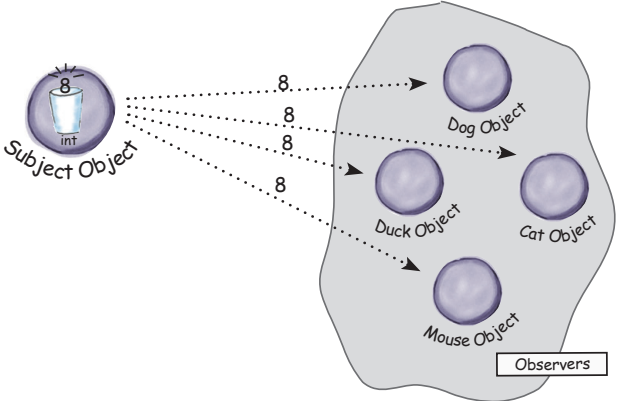
The Duck object is now an official observer.

Duck is psyched...he's on the list and is waiting with great anticipation for the next notification so he can get an int.



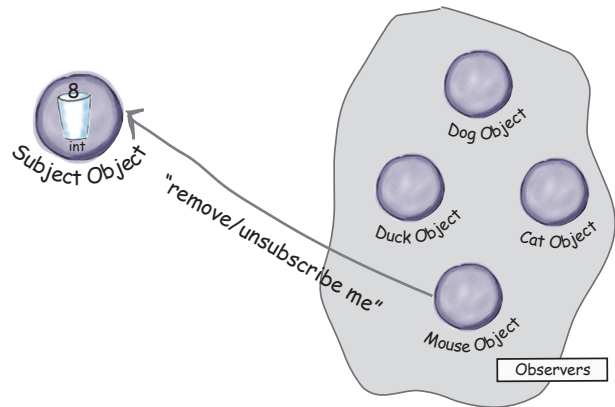
The Subject gets a new data value!

Now Duck and all the rest of the observers get a notification that the Subject has changed.



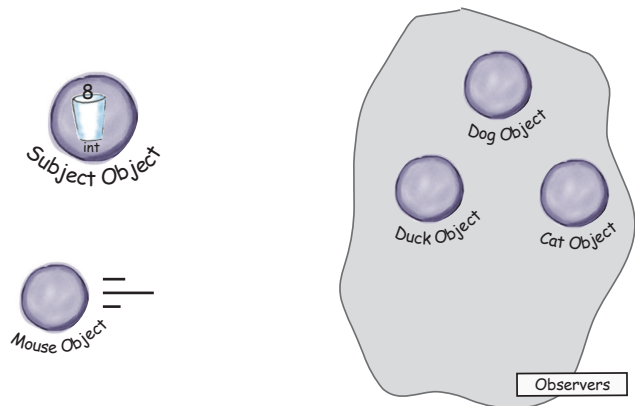
The Mouse object asks to be removed as an observer.

The Mouse object has been getting ints for ages and is tired of it, so he decides it's time to stop being an observer.



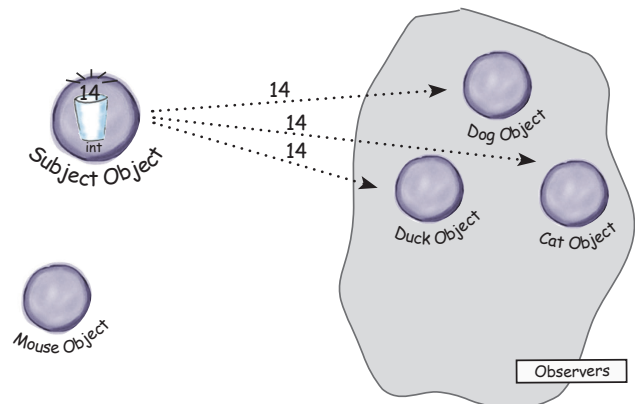
Mouse is outta here!

The Subject acknowledges the Mouse's request and removes him from the set of observers.



The Subject has another new int.

All the observers get another notification, except for the Mouse who is no longer included. Don't tell anyone, but the Mouse secretly misses those ints... maybe he'll ask to be an observer again some day.





Five-minute drama: a subject for observation

In today's skit, two enterprising software developers encounter a real live head hunter...

This is Lori. I'm looking for a Java development position. I've got five years of experience and...



1

Software Developer #1

Uh, yeah, you and everybody else, baby. I'm putting you on my list of Java developers. Don't call me, I'll call you!



2

Headhunter/Subject

Hi, I'm Jill. I've written a lot of enterprise systems. I'm interested in any job you've got with Java development.



3

Software Developer #2

I'll add you to the list—you'll know along with everyone else.



4

Subject

- 5** Meanwhile, for Lori and Jill life goes on; if a Java job comes along, they'll get notified. After all, they are observers.



Two weeks later...



Jill's loving life, and no longer an observer. She's also enjoying the nice fat signing bonus that she got because the company didn't have to pay a headhunter.

But what has become of our dear Lori? We hear she's beating the headhunter at his own game. She's not only still an observer, she's got her own call list now, and she is notifying her own observers. Lori's a subject and an observer all in one.



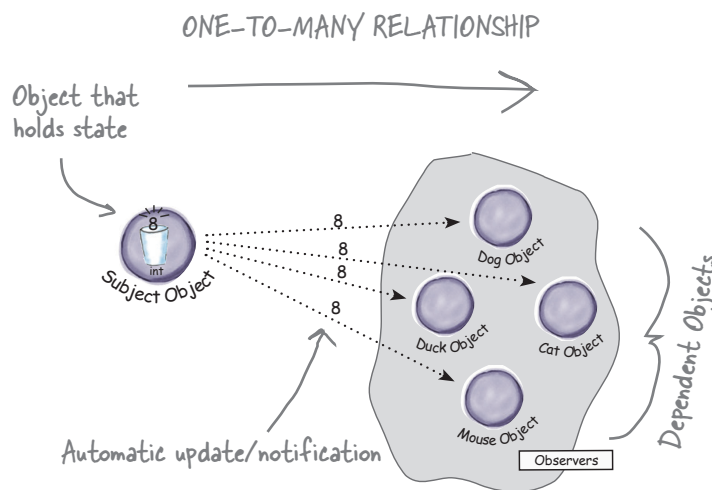
The Observer Pattern defined

A newspaper subscription, with its publisher and subscribers, is a good way to visualize the pattern.

In the real world, however, you'll typically see the Observer Pattern defined like this:

The Observer Pattern defines a one-to-many dependency between objects so that when one object changes state, all of its dependents are notified and updated automatically.

Let's relate this definition to how we've been thinking about the pattern:



The subject and observers define the one-to-many relationship. We have *one subject*, who notifies *many observers* when something in the subject changes. The observers *are dependent* on the subject—when the subject's state changes, the observers are notified.

As you'll discover, there are a few different ways to implement the Observer Pattern, but most revolve around a class design that includes Subject and Observer interfaces.

The Observer Pattern defines a one-to-many relationship between a set of objects.

When the state of one object changes, all of its dependents are notified.

The Observer Pattern: the Class Diagram

Let's take a look at the structure of the Observer Pattern, complete with its Subject and Observer classes. Here's the class diagram:

Here's the Subject interface. Objects use this interface to register as observers and also to remove themselves from being observers.

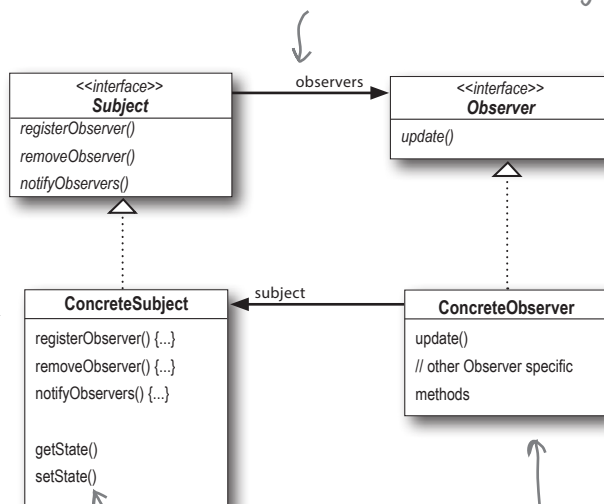
Each subject can have many observers.

All potential observers need to implement the Observer interface. This interface has just one method, `update()`, that is called when the Subject's state changes.

A concrete subject always implements the Subject interface. In addition to the register and remove methods, the concrete subject implements a `notifyObservers()` method that is used to update all the current observers whenever state changes.

The concrete subject may also have methods for setting and getting its state (more about this later).

Concrete observers can be any class that implements the Observer interface. Each observer registers with a concrete subject to receive updates.



there are no Dumb Questions

Q: What does this have to do with one-to-many relationships?

A: With the Observer Pattern, the Subject is the object that contains the state and controls it. So, there is ONE subject with state. The observers, on the other hand, use the state, even if they don't own it. There are many observers, and they rely on the Subject to tell them when its state changes. So there is a relationship between the ONE Subject to the MANY Observers.

Q: How does dependence come into this?

A: Because the subject is the sole owner of that data, the observers are dependent on the subject to update them when the data changes. This leads to a cleaner OO design than allowing many objects to control the same data.

Q: I've also heard of a Publish-Subscribe Pattern. Is that just another name for the Observer Pattern?

A: No, although they are related. The Publish-Subscribe pattern is a more complex pattern that allows subscribers to express interest in different types of messages and further separates publishers from subscribers. It is often used in middleware systems.